

Raspberry Pi 4B PVVX Bluetooth Thermometer MQTT Bridge

Objective

Replace an ESP32/Tasmota BLE gateway with the Raspberry Pi 4B's built-in Bluetooth adapter, while preserving the existing MQTT topic structure used by downstream applications.

System Configuration

Hardware

- Raspberry Pi 4B (preferably not using the 2.4Ghz wi-fi channel)
- Xiaomi LYWSD03MMC thermometers
- PVVX firmware installed on thermometers

Software

- Raspberry Pi OS Trixie
 - Mosquitto MQTT broker
 - Python virtual environment
 - Bleak Bluetooth library
 - Paho MQTT library
-

Bluetooth Troubleshooting

Initially Bluetooth appeared unavailable.

Symptoms:

```
bluetoothctl power on
Failed to set power on: org.bluez.Error.Failed
```

Investigation revealed Bluetooth was soft-blocked.

Check status:

```
rfkill list
```

Example faulty output:

```
0: hci0: Bluetooth
    Soft blocked: yes
```

Fix:

Fixing Bluetooth Not Starting on Boot

After reboot the script was failing with `BleakBluetoothNotAvailableError` because the Bluetooth adapter was powered off. Root cause: `AutoEnable` was commented out in `main.conf`, and the adapter was soft-blocked by `rfkill`.

Symptoms Found

Check	Result	Meaning
<code>hciconfig -a</code>	DOWN	Adapter not powered
<code>rfkill list</code>	Soft blocked: yes	Kernel is blocking BT
<code>main.conf</code>	<code>#AutoEnable=true</code>	Daemon not powering adapter on start

Fix: Enable AutoEnable in bluetoothd

i This is the permanent fix – uncomment `AutoEnable` in `/etc/bluetooth/main.conf`.

Check current value:

```
grep -i autoenable /etc/bluetooth/main.conf
```

Uncomment it:

```
sudo sed -i 's/#AutoEnable=true/AutoEnable=true/' /etc/bluetooth/main.conf
```

Verify:

```
grep -i autoenable /etc/bluetooth/main.conf
```

Should show: `AutoEnable=true`

Verify:

```
bluetoothctl
power on
scan on
```

Both PVVX sensors became visible.

Python Environment

Trixie enforces PEP 668 and prevents direct pip installation.

Attempting:

```
pip install bleak paho-mqtt
```

produced:

```
externally-managed-environment
```

Solution:

Install venv support:

```
sudo apt install python3-venv python3-full
```

Create environment:

```
python3 -m venv ~/venvs/pvvx # NB = /home/maxair/venvs/pvvx/
```

Activate:

```
source ~/venvs/pvvx/bin/activate
```

Install packages:

```
pip install bleak paho-mqtt
```

MQTT Requirements

Existing applications expected:

Topic:

```
tele/tasmota/SENSOR
```

Payload example:

```
{  
  "Time": "2026-05-29T15:00:00",  
  "ATCe284f9": {  
    "Temperature": 25.4,  
    "Humidity": 49,  
  }  
}
```

```
    "Battery": 79
  },
  "ATC094e73": {
    "Temperature": 22.9,
    "Humidity": 51,
    "Battery": 83
  }
}
```

The Python bridge was modified to emulate Tasmota SENSOR messages exactly.

MQTT Authentication

Broker uses authentication.

Test subscription:

```
mosquitto_sub -t '#' -v -u admin -P pihome
```

Successful subscription confirms broker functionality.

Systemd Service

Create Service file: `/etc/systemd/system/pvvx-ble.service`

[Unit]

Description=PVVX BLE to MQTT Bridge

After=bluetooth.service mosquitto.service

Requires=bluetooth.service

[Service]

ExecStart=/home/maxair/venvs/pvvox/bin/python /home/maxair/pvvox_ble_mqtt.py

WorkingDirectory=/home/maxair

Restart=always

RestartSec=5

User=maxair

Environment=PYTHONUNBUFFERED=1

[Install]

WantedBy=multi-user.target

Save as:

`/etc/systemd/system/pvvox-ble.service`

Enable:

```
sudo systemctl daemon-reload
sudo systemctl enable pvvx-ble
sudo systemctl start pvvx-ble
```

Watch logs:

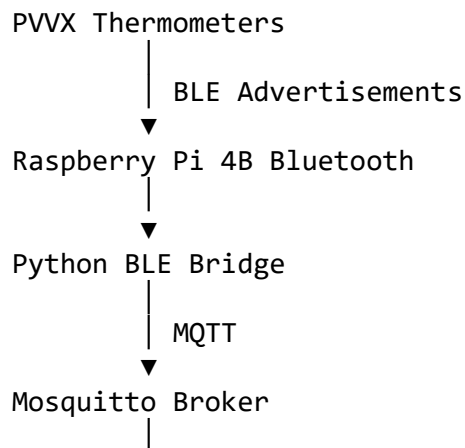
```
journalctl -u pvvx-ble -f
```

Operational Characteristics

The final bridge:

- Uses Raspberry Pi built-in Bluetooth
 - Receives PVVX advertisements passively
 - Does not connect or pair with sensors
 - Publishes every 55 seconds
 - Publishes valid JSON
 - Publishes to:
 tele/tasmota/SENSOR
 - Reports:
 - Temperature
 - Humidity
 - Battery percentage
-

Final Architecture



Lessons Learned

1. Bluetooth was functional but soft-blocked by rfkill.
2. Mosquitto failures were caused by a duplicate configuration file.
3. Debian Trixie requires Python virtual environments for third-party packages.
4. Maintaining the original Tasmota MQTT topic structure avoided downstream changes.
5. Passive BLE scanning is sufficient for PVVX sensors.
6. Once stable, avoid unnecessary changes to the bridge.

Final Python Script: pvvx_ble_mqtt.py

Located in the /home/maxair directory.

```
#!/usr/bin/env python3
```

```
import asyncio
import struct
import time
from datetime import datetime
from bleak import BleakScanner
import paho.mqtt.client as mqtt
import json
```

```
# ===== MQTT =====
MQTT_HOST = "127.0.0.1" # Use your own content for these 5 lines
MQTT_PORT = 1883
MQTT_USER = "admin"
MQTT_PASS = "pihome"
```

```
TOPIC = "tele/tasmota/SENSOR" # Once tested, I deleted tasmota here AND in MaxAir.
```

```
# ===== BEHAVIOUR =====
PUBLISH_INTERVAL = 55 # seconds, your choice.
```

```

state = {}
last_publish = 0

# ===== MQTT CLIENT =====
client = mqtt.Client()
client.username_pw_set(MQTT_USER, MQTT_PASS)
client.connect(MQTT_HOST, MQTT_PORT, 60)
client.loop_start()

# ===== MAC -> Tasmota-style ATC key =====
def mac_to_atc(mac: str):
    return "ATC" + mac.replace(":", "")[-6:].lower()

# ===== UPDATE STATE =====
def update_state(mac, parsed):

    key = mac_to_atc(mac)

    state[key] = {
        "Temperature": parsed["Temperature"],
        "Humidity": parsed["Humidity"],
        "Battery": parsed["Battery"],
        "LastUpdate": time.time()
    }

# ===== PUBLISH FULL TASMOTA SENSOR FRAME =====
def publish():
    global last_publish

    payload = {
        "Time": datetime.now().isoformat(),
    }

    # strip internal metadata before publishing
    for k, v in state.items():
        payload[k] = {
            "Temperature": v["Temperature"],
            "Humidity": v["Humidity"],

```

```

        "Battery": v["Battery"]
    }

    client.publish(TOPIC, json.dumps(payload), qos=0, retain=False)
    print("PUBLISHED:", payload)

    last_publish = time.time()

# ===== BLE CALLBACK =====
def callback(device, advertisement_data):

    service_data = advertisement_data.service_data

    if not service_data:
        return

    for uuid, data in service_data.items():

        # Environmental sensing UUID
        if uuid.lower() == "0000181a-0000-1000-8000-00805f9b34fb":

            try:
                # PVVX custom format — little-endian, hundredths
                temp = int.from_bytes(data[6:8], 'little', signed=True) / 100
                hum = int.from_bytes(data[8:10], 'little') / 100
                batt = data[12] # battery % at offset 12, after 2-byte voltage at 10-11

                parsed = {
                    "Temperature": round(temp, 1),
                    "Humidity": round(hum, 1),
                    "Battery": batt
                }

                update_state(device.address, parsed)
                print(device.address, parsed)

            except Exception as e:
                print("PARSE ERROR:", e)

```



```
# ===== MAIN LOOP =====
async def main():
    global last_publish

    scanner = BleakScanner(callback)
    await scanner.start()

    while True:
        await asyncio.sleep(5)

        # publish every 55 seconds regardless of sensor updates
        if time.time() - last_publish > PUBLISH_INTERVAL:
            if state:
                publish()

if __name__ == "__main__":
    asyncio.run(main())
```